

AULA 10

Do laço às entidades

O Pong no último laço — e por que o próximo passo é orientação a objetos

HOJE

Duas partes

1. Adaptar o Pong para o laço de passo fixo — e ver quanto mudou.
2. Por que o próximo assunto é orientação a objetos.

A segunda parte é preparação de terreno: Update Method e Componentes vêm depois, e os dois pedem entidades.

O que pedimos — e o que não pode mudar

Muda:

```
o laço: acumulador, while interno, update(PASSO) e a trava
```

Não muda:

```
as velocidades continuam em px por segundo  
nada dentro de inputs(), update() e draw()
```

Especificar antes de pedir. Sem isso, o prompt vira “conserta aí”.

O laço que temos

```
while rodando:  
    dt = clock.tick(60) / 1000  
    inputs(dt)  
    update(dt)  
    draw()
```

O tamanho do passo é o que a máquina deu.

O laço que queremos

```
PASSO = 1 / 60
acumulador = 0

while rodando:
    acumulador += min(clock.tick(240) / 1000, 0.25)
    inputs()
    while acumulador >= PASSO:
        update(PASSO)
        acumulador -= PASSO
    draw()
```

tick(240) em vez de tick(60): o freio do relógio sai do caminho da física, mas o laço não vira um moinho.

Três linhas no laço. Nada nas funções.

<code>inputs()</code>	igual	só perdeu o parâmetro <code>dt</code>
<code>update()</code>	igual por dentro	recebe <code>PASSO</code> em vez de <code>dt</code>
<code>draw()</code>	igual	continua uma vez por volta
<code>velocidades</code>	iguais	300 px por segundo, como antes
<code>colisão</code>	igual	<code>abs()</code> e reposicionamento, da aula 08

Por que saiu tão barato? Porque o código já estava separado em `inputs`, `update` e `draw`.

O laço está resolvido. O código, não.

O Pong tem doze variáveis soltas: `bola_x`, `bola_y`, `bola_vx`, `bola_vy`, `jogador_y`, `oponente_y`, `placar_jogador`, `placar_oponente`...

Todas visíveis para todo mundo, o tempo inteiro.

E o `update()` precisa conhecer todas elas para funcionar.

E se o jogo tiver duas bolas?
E três raquetes? E dez itens caindo?

A linha que gostaríamos de escrever

```
for entidade in entidades:  
    entidade.update(PASSO)
```

Uma bola, dez bolas, vinte itens: o laço não muda.

Para escrever isso, duas coisas precisam existir.

Duas condições para aquela linha funcionar

1 · cada coisa carrega o seu estado
a bola guarda a própria posição e
velocidade — em vez de quatro
variáveis soltas no arquivo

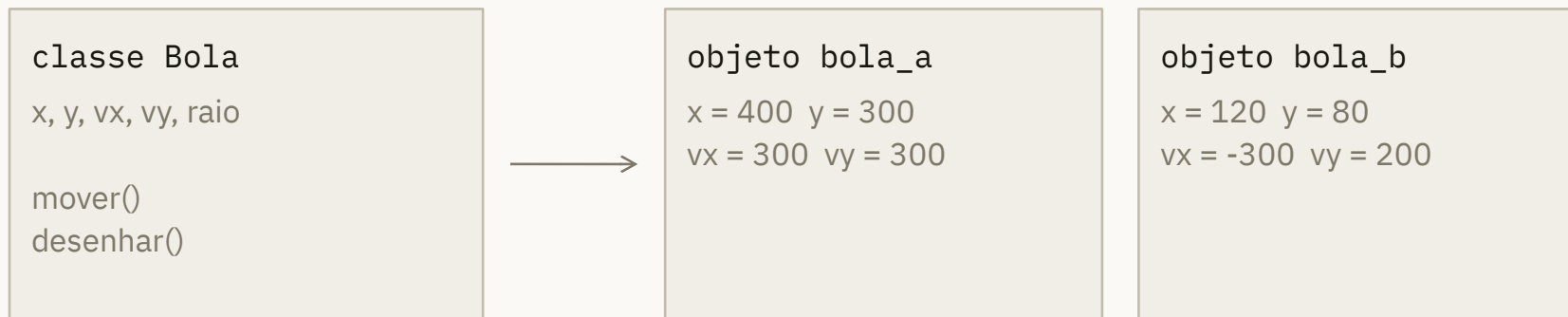
objeto

2 · todas respondem ao mesmo chamado
update(PASSO) funciona para bola,
raquete e item — cada uma faz
a sua coisa

polimorfismo

E o nome do padrão que usa as duas, no nosso livro, é Update Method.

A planta e as casas



A classe descreve o que toda bola tem e faz. O objeto é uma bola concreta, com os seus próprios valores. Duas bolas, dois conjuntos de dados, um só código.

Estado e comportamento no mesmo lugar

hoje

```
bola_x, bola_y, bola_vx...
```

```
def update(dt):  
    # move a bola  
    # move o oponente  
    # testa colisões  
    # conta pontos
```

com objetos

```
class Bola:  
    def __init__(self):  
        self.x = 400  
        self.vx = 300  
  
    def update(self, dt):  
        self.x += self.vx * dt
```

A bola sabe onde está e sabe se mover. Ninguém de fora precisa saber como.

O que é comum fica em um lugar só



As duas são raquetes e compartilham quase tudo. Muda só quem decide o movimento.

O mesmo chamado, comportamentos diferentes

```
for entidade in entidades:  
    entidade.update(PASSO)
```

Bola

anda e quica

RaqueteDoJogador

lê o teclado

RaqueteDoOponente

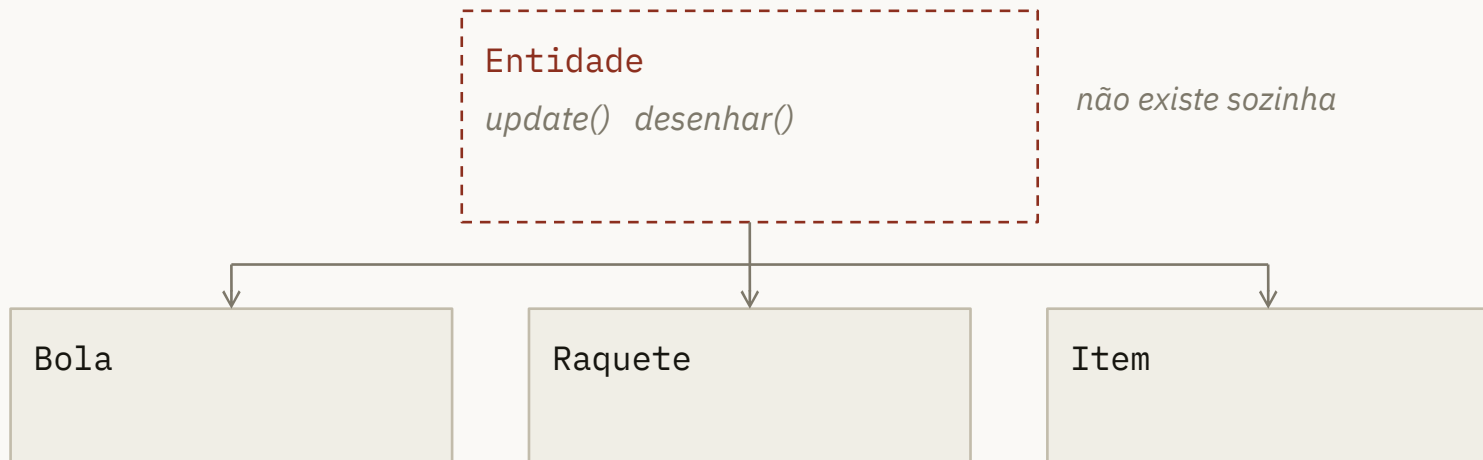
segue a bola

Quem chama não precisa saber quem é. Cada uma sabe o que fazer quando chamada.

O que garante que toda entidade da lista tem um update?

Se uma delas não tiver, o laço quebra no meio do jogo.

O contrato: o que toda entidade promete ter



Declara quais métodos existem, sem dizer como cada um faz.

Em C++, o contrato é obrigatório

```
class Entidade {  
public:  
    virtual void update(float dt) = 0;    // sem corpo  
    virtual void desenhar() = 0;  
};  
  
std::vector<Entidade*> entidades;          // um tipo só na lista
```

Sem a classe base, o código não compila: o vetor exige um tipo comum, e o virtual é o que manda decidir o método em tempo de execução.

As duas exigências desaparecem

```
class Bola:
    def update(self, dt): ...

entidades = [bola, jogador, oponente]    # sem tipo em comum
for e in entidades:
    e.update(PASSO)                        # funciona
```

A lista aceita qualquer objeto, e a chamada já é resolvida na hora.

Se tem update, serve. Ninguém precisa herdar de nada.

Então a classe abstrata
não serve para nada em Python?

Ela muda de função

Em C++ é exigência do compilador. Em Python, é documentação executável:

```
from abc import ABC, abstractmethod

class Entidade(ABC):
    @abstractmethod
    def update(self, dt): ...
```

Escrito assim, o Python reclama se uma filha esquecer o método.

O contrato deixa de morar só na cabeça de quem escreveu.

Metade do cuidado do livro é com memória

ponteiros

Entidade* , new , delete , destrutores virtuais — em Python não existem

layout de memória

o capítulo de otimização organiza objetos para a CPU ler rápido; em Python isso não está ao nosso alcance

herança × composição

em C++ a escolha tem custo de desempenho; em Python é quase só clareza

Ler o livro sabendo disso evita copiar cerimônia que não paga nada aqui.

O Pong vira entidades

A bola, as duas raquetes e o placar deixam de ser variáveis soltas.

E aquela linha do for deixa de ser slide e vira código.

Para casa

terminar o passo fixo no seu Pong, se não terminou em aula

ler o capítulo Update Method — agora ele tem contexto